PicoBoot-WiFi: architecture

Project	PicoBoot-WiFi: PicoBoot-DX for the Pico 2 W, with over-the-air firmware and payload updates
Status	Approved v1.6, 2026-11-11; draft v1.7 (D24, the release) for approval at the end of phase 5
Owner	Remko Kleinjan
Research	docs/research/

Status

Version	Date	What changed and why
v0.1	2026-11-08	First draft. Builds on the 2026-11-07 OTA research and the owner's layout proposal (slots A/B 256K, payload 1 MB, Wi-Fi 500K). New research (`docs/research/`) changes three things: the payload area must stay 1.5 MB (Swiss r2119 is 1.29 MiB); the Pico 2 W has 4 MB of flash; and a partition table would break USB payload installs, so this draft proposes a loader of our own instead of the boot ROM's A/B.
v0.2	2026-11-08	The owner answered four rounds of questions. Fifteen items became decisions (D6, D7, D9, D11, D12, D14–D17, D19–D22, D27, D28). Home Wi-Fi (D21) moves out of the first release and stays an option for later releases, so it leaves the phase plan. New proposals: the license (D29) and the access point's security details (D30). Proposals D4, D5, D8, D10, D13, D18, D23–D26, D29, D30 and the phase plan in D28 wait for approval with the rest of the design.
v1.0	2026-11-08	The owner approved v0.2 with all its proposals (”ok, go ahead”). Every proposal is now a decision; nothing else changed. Step 1 of the design cycle is closed; the implementation plan (`docs/implementation-plan.md`) follows.
v1.1	2026-11-08	The implementation plan's design points DP1–DP5, approved with the plan, become D31–D35. The owner has a current meter but no oscilloscope or logic analyzer, and one DOL-001, so: the Pico measures its own boot timing (D31, changes D10); a minimal Wi-Fi report page comes early (D32, changes D28); the rail is checked with a stress test instead of a scope (D33, changes D28 phase 0); the DOL-001 is the test console (D34); and there are no pre-releases (D35, changes D24).
v1.2	2026-11-08	Approved by the owner. Phase 0 results (phase0-results.md). The owner decided three changes: guard the 224 skipped CS pulses against glitches with a CS pull-up and a clock-qualified count (D36), after 2 blue screens in 56 cold boots were traced to a false CS edge at the console's power-up; keep the last flash sector free, because USB installs write picotool's RP2350-E10 block there (D37, changes D6); and recommend 5 V wiring, because the console's 3.3 V at the Pico is 3.15–3.21 V (D38). Also noted: HAD_POR survives warm reboots, so D8's marker is required (confirmed); the boot margin is about 166 ms (D10); the risks table is updated.
v1.3	2026-11-08	Approved by the owner at the end of phase 1. During phase 1, for step 1.10's many console boots, the owner chose a boot log in the loader region (D39): the loader region shrinks to 8K (the loader is 2.7K), and the freed 8K keep up to the last 32 boot reports. Step 1.11 sets two values from the phase 1 console tests (phase1-results.md), as the owner chose: the boot path may add at most 10 ms over PicoBoot-DX (D10; the loader costs 3–5 ms), and the injection keeps 250 MHz with flash divider 3 (D27; 87 console boots, cold and warm, without a fault). The 225th-transfer edge check was proven on hardware (2 power-up glitches survived); the phase 1 code review found one theoretical gap, which the owner chose to accept as a known limit rather than change the proven programs (D36). For approval at the end of phase 1.

Version	Date	What changed and why
v1.4	2026-11-09	Approved by the owner at the end of phase 2. Phase 2, step 2.1: the settings, their values and defaults (D40). The owner chose channel 6 as the default and named Tx power levels (low, medium, high, max) instead of a dBm number. Step 2.3: how a settings change takes effect (D41). The owner chose an access-point restart at once, a trial for Wi-Fi changes until a client joins, and a write-only password. Step 2.6: the owner set the Tx power levels to 5, 10 and 15 dBm and the chip's limit (D40), and the stalled-injection LED to blink for as long as Wi-Fi is on (D26).
v1.5	2026-11-10	Approved by the owner at the end of phase 3. Phase 3, step 3.1: the update file format (update-format.md). The owner chose to always carry the Wi-Fi blob in the release file, which drops D14's copy between the Wi-Fi areas, and to allow downgrades with a warning (D11). The header names the layout and the lowest loader version, since OTA can't update the loader (D11). Step 3.4: D12 as built; the owner chose 3 crashes for the crash loop, and a fallback for good.
v1.6	2026-11-11	Approved by the owner at the end of phase 4. Phase 4: payload uploads (D42). The owner chose names they pick (which brings boot control format 2 and loader version 3, one USB install), uploads into payload 2 while the built-in boots, no deleting the chosen payload, and backups without the Wi-Fi password. D17: a USB install is recorded after a complete injection (or a choice, rename or upload over Wi-Fi), so the console's power-on shows it. D43: a loader stub for ordinary `.dol` files, for a later release. The phase 4 review: a wider fingerprint, no update to a firmware from before boot control format 2, Wi-Fi 1.5 s later on an ”original IPL once” boot.
v1.7 draft	2026-11-11	Phase 5: how a release is made (D24): a draft release by CI on a version tag, with the CLI and checksums; a release's version without the git hash. The security review (D19): what the password gate means, and DNS-rebinding reads of the GETs accepted by the owner.

How to read the decisions

Every decision has a number (D1, D2, …) that stays the same in later versions. Each one is one of three kinds:

- **Decision:** made by the owner.
- **Proposal:** waiting for the owner's approval.
- **Open:** a question that needs an answer, often from a hardware test.

Summary

#	Topic	Kind	In short
D1	Board	Decision	Pico 2 W only
D2	Goal	Decision	PicoBoot-DX's job, plus OTA of firmware and payload
D3	Code base	Decision	New project; PicoBoot-DX code may be reused
D4	Compatibility	Decision	Drop-in for PicoBoot-DX: same wiring, payload format and address
D5	Non-goals	Decision	No other boards, no internet update check, no Bluetooth, no HTTPS
D6	Flash layout	Decision	256K slots, 2×240K Wi-Fi, payload 1.5 MB at `0x80000`, second payload, all 4 MB
D7	Slot switching	Decision	Own loader with `rom_chain_image()`, no partition table
D8	When to inject	Decision	Only on a real power-on
D9	Wi-Fi start	Decision	Only after the injection has finished
D10	Boot-time budget	Decision	At most 10 ms over PicoBoot-DX before the PIO is ready; the loader path costs 3–5 ms

#	Topic	Kind	In short
D11	Update file	Decision	One file with the firmware, optionally the Wi-Fi blob; SHA-256, no signing
D12	Trial and fallback	Decision	On trial until a good power-on boot or *Keep*; a crash falls back
D13	Switching firmware	Decision	Free switching between two good slots
D14	Wi-Fi blob	Decision	Outside the slots, one copy per slot, at fixed addresses
D15	Payload slots	Decision	Two payload slots; you choose which one boots
D16	Payload formats	Decision	Payload UF2 (any family) and raw payload on the device; `.dol` converted in the browser and the CLI
D17	USB payload installs	Decision	A payload installed over USB always boots next
D18	Web stack	Decision	Access point, captive portal, lwIP httpd, small static UI, JSON API
D19	Wi-Fi security	Decision	WPA2, fixed documented default password, banner until changed
D20	Wi-Fi on-time	Decision	Off after 10 minutes without a client; a setting
D21	Home Wi-Fi	Decision	Not in the first release; an option for later releases
D22	Features	Decision	Status, settings, OTA, diagnostics, ”original IPL once”, backups, CLI tool
D23	Storage	Decision	Boot control and settings in two-sector records that survive power loss
D24	Releases and tools	Decision	USB install UF2, update file, Python tools
D25	Build and test	Decision	SDK 2.3.1, CI, PIO simulator, host tests for the loader logic
D26	LED	Decision	Dark during the injection (D9), status after
D27	Clock and flash speed	Decision	250 MHz during the injection, flash divider 3 (83 MHz); both build settings
D28	Phases	Decision	Hardware checks first, then phases 1–5 as listed
D29	License	Decision	GPL-2.0-only, as PicoBoot-DX
D30	AP security details	Decision	WPA2-AES only, default password `picoboot`, SSID from the chip ID
D31	Boot-timing measurement	Decision	The Pico measures the margin itself with a passive PIO counter
D32	Early readout	Decision	A minimal Wi-Fi report page in phases 0 and 1
D33	Rail check	Decision	Average current plus a stress test instead of a scope
D34	Test console	Decision	DOL-001 with SD2SP2 and HDMI adapter; DOL-101 untested
D35	Releases	Decision	No pre-releases; test builds 0.N.0-dev; first release 1.0.0
D36	CS glitch protection	Decision	CS pull-up, and a clock-qualified count of the 224 skipped pulses
D37	Last flash sector	Decision	Kept free; payload 2 is 4K shorter
D38	Wiring	Decision	README recommends 5 V to VSYS; 3V3 stays supported
D39	Boot log	Decision	Up to the last 32 boot reports in the loader region's upper 8K
D40	Settings	Decision	Their values and defaults: channel 6 (1–11), Tx power in named levels (medium), country worldwide
D41	Settings changes	Decision	At once; Wi-Fi changes restart the AP and stay on trial until a client joins; the password is write-only
D42	Payload uploads	Decision	Names you choose; into payload 2 while the built-in boots; the chosen payload can't be deleted; backups without the password; boot control format 2 with loader 3
D43	Ordinary `.dol` files	Decision	Not in the first release: a loader stub in the payload, so that `.dol` files not built for the IPL's address boot too; an option for later releases

1. Goal and scope

D1 (Decision): Pico 2 W only. RP2350A, CYW43439, 4 MB flash. The Pico W, Pico and Pico 2 stay with PicoBoot-DX.

D2 (Decision): the same job as PicoBoot-DX, plus over-the-air updates.

- Inject the IPL payload at power-on, exactly as PicoBoot-DX does.

- Add a Wi-Fi access point with a web page to update the firmware and the payload.

D3 (Decision): a new project, built from the ground up. PicoBoot-DX code may be reused where it fits, but the design isn't bound to it.

D4 (Decision): a drop-in for PicoBoot-DX on a Pico 2 W.

- **Wiring:** the same, CS GP4, CLK GP5, DATA GP6. Power through 3V3, or 5 V through a diode to VSYS, as today.
- **Payloads:** the same format (`IPLBOOT` header, `PICO` footer) at the same address, `0x10080000`. Every existing PicoBoot payload UF2 keeps installing over USB.
- **Fallback:** the same built-in gekkoboot, used when there is no valid payload.
- **Upgrade from PicoBoot-DX:** one install UF2 over USB. It leaves the installed payload alone.

D5 (Decision): non-goals.

- other boards;
- checking for updates on the internet (needs HTTPS and a server);
- Bluetooth;
- HTTPS for the web page;
- re-injecting after the console's reset button (PicoBoot-DX doesn't either).

D38 (Decision): recommend the 5 V wiring.

- **Measured in phase 0:** with the 3V3 wiring, the console's 3.3 V at the Pico is 3.21 V with the radio off and 3.15 V with it on. That's below the 3.2 V where the CYW43439 guarantees full RF performance.
- **Still worked:** range (−42 dBm at 5 m) and a 10-minute full-load stress test were fine anyway.
- **README:** it recommends 5 V through a Schottky diode to VSYS. The Pico's own regulator then feeds the radio from 5 V.
- **3V3 wiring:** stays supported.

2. Constraints

These follow from the research and bind every decision below.

#	Constraint	Source
C1	**Injection timing:** the PIO has to count CS pulses from the console's very first IPL read. Nothing slow may run before the PIO starts, and nobody has measured how much margin there is.	hardware.md §3
C2	**No second injection:** a reboot while the console is on (update, watchdog, rollback) must not inject again. The PIO would count later EXI reads and drive the data line in the middle of a game.	ota-research.md
C3	**Payload address:** existing payload UF2s write `0x10080000` with the rp2350-arm-s family (and rp2040 blocks, which the RP2350 ignores). They must keep landing at flash offset `0x80000`.	boot-rom.md §5
C4	**Payload size:** up to 1.5 MB. Swiss r2119 is 1.29 MiB, and the console reads at most about 1.69 MiB.	ecosystem.md §2
C5	**Power:** the radio runs from the console's 3.3 V rail (or from VSYS with 5 V wiring). Tx bursts can reach 320 mA at full power.	hardware.md §1
C6	**Flash access:** while flash is erased or programmed, XIP reads fault. During the injection the payload DMA reads flash, so nothing may write flash then.	boot-rom.md §8

3. Flash layout

D6 (Decision): 256K slots, two 240K Wi-Fi areas, payload 1.5 MB at `0x80000`, a second payload, all 4 MB used.

- **Origin:** the owner's first proposal (slots A/B 256K, payload 1 MB, Wi-Fi 500K), adjusted after the research.
- **Payload:** 1.5 MB, because 1 MB is 301 KB too small for Swiss r2119 (C4).

- **Upper 2 MB:** used, since the Pico 2 W has 4 MB of flash.
- **The owner chose this over the swap with 240K slots and 256K Wi-Fi areas.**

Region	Flash offset	Size	Holds	Written by
Loader	`0x000000`–`0x002000`	8K	The loader (D7), 2.7K	USB install only
Boot log	`0x002000`–`0x004000`	8K	Up to the last 32 boot reports (D39)	Firmware
Boot control	`0x004000`–`0x006000`	8K	Active and pending slot, payload choice (D23)	Firmware
Settings	`0x006000`–`0x008000`	8K	Wi-Fi and user settings (D23)	Firmware
Wi-Fi A	`0x008000`–`0x044000`	240K	CYW43 blob for slot A (D14)	Install, OTA
Wi-Fi B	`0x044000`–`0x080000`	240K	CYW43 blob for slot B (D14)	OTA
Payload 1	`0x080000`–`0x200000`	**1.5 MB**	The payload at the PicoBoot address (C3)	USB, OTA
Slot A	`0x200000`–`0x240000`	256K	Firmware	Install, OTA
Slot B	`0x240000`–`0x280000`	256K	Firmware	OTA
Payload 2	`0x280000`–`0x3FF000`	1.5 MB − 4K	Second payload (D15)	OTA
Reserved	`0x3FF000`–`0x400000`	4K	Nothing: stock RP2350 UF2s write picotool's E10 block here (D37)	—

Why it fits:

- **Slots:** the measured stand-in firmware is 165 KB, the built-in gekkoboot included, which leaves about 90 KB for the web UI and features (sizes.md).
- **Wi-Fi areas:** the blob is 225,240 bytes (Wi-Fi only) or 232,596 bytes (with Bluetooth), against 245,760 available.
- **Payload 1:** stays at `0x80000`, below the 2 MB line, where PicoBoot-DX had it.

Margins and risks:

- **Wi-Fi areas:** 13–20 KB to spare. A much larger future blob would need a layout change, which means a USB install.
- **No partition table:** the layout lives only in the loader and the firmware, never in flash. That's what keeps USB payload UF2s working (C3).

D39 (Decision): a boot log in the loader region (changes D6).

- **Why:** step 1.10's console tests need 60+ power-ons. Phase 0 showed that a log read over Wi-Fi makes that practical: one phone connection for a whole series.
- **Where:** the loader region shrinks to 8K, since the loader is 2.7K. The freed 8K (`0x002000`–`0x004000`), two sectors, keep a ring of 256-byte entries: up to the last 32 boot reports. When the ring wraps it erases a sector of 16, so it holds 17 to 32.
- **When it's written:** every boot, warm ones included, after the injection and before Wi-Fi starts, so nothing reads flash in the background (C6).
- **Torn entries:** a torn entry in the middle of a sector makes the ring move on to the next sector, so the newest entries are never erased (phase 0 review finding 2).
- **After release:** it stays as part of the diagnostics (D22).

D37 (Decision): keep the last flash sector free (changes D6).

- **Found in phase 0:** a USB install of a picotool-built RP2350 UF2 also writes picotool's RP2350-E10 workaround block (256 × `0xEF`, absolute family, address `0x10FFFF00`).

  - Without a partition table the boot ROM accepts it.
  - The address wraps on the 4 MB flash, so the block lands in its last page.
  - This holds for PicoBoot-DX, pico-examples and our own SDK builds alike.
- **Payload 2** ends at `0x3FF000`: 1,568,768 bytes, more than Swiss r2119's 1,349,956. A larger payload can only go into payload 1.
- **Our own install UF2** (D24) leaves the E10 block out. E10 only matters with a partition table, which we don't use.

4. Boot

D7 (Decision): a loader of our own with `rom_chain_image()`, and no partition table.

- **What it is:** a small image at flash offset 0 that the boot ROM starts as a normal flash image.
- **What it does:** reads boot control, picks a slot and hands over with `rom_chain_image()`. The ROM maps the chosen slot to `0x10000000` through address translation, so slot A and slot B run the same binary.
- **Why not the boot ROM's A/B:**
  - **USB payloads:** with a partition table the ROM routes every non-absolute UF2 into a partition, so payload UF2s would no longer install over USB (C3).
  - **Switching:** the ROM's A/B always erases one image when you switch.
  - **Install:** it takes three UF2 files.
  - **A2 chips:** they also have erratum E10, which breaks drag-and-drop UF2 installs once a partition table exists.
- **What it costs:**
  - one extra ROM image scan before the firmware starts (D10);
  - our own fallback logic (D12);
  - a small hook so the Wi-Fi blob loads from a fixed address (D14);
  - the loader itself can only be updated over USB.
- **Rules for the loader:**
  - tiny;
  - no clock setup;
  - no flash writes;
  - no Wi-Fi;
  - all its decisions in one pure function, tested on the host (D25).

D8 (Decision): inject only on a real power-on.

- **The rule:** a boot is a power-on boot only when both hold:
  - POWMAN `CHIP_RESET.HAD_POR` is set;
  - our marker in a POWMAN scratch register is missing.
- **Why both:** POWMAN scratch survives watchdog and software reboots and is cleared only at power-on, brown-out or RUN. So the pair still works if HAD_POR turns out to be sticky (to be checked, boot-rom.md §7).
- **Brown-out:** a brown-out (HAD_BOR) does **not** count as a power-on. If the Pico browns out during a Tx burst while the console keeps running, a second injection would hit a running console (C2).
- **What it controls:** only the power-on decision. Everything else that has to survive a warm reboot goes in POWMAN scratch: the session marker, the boot reason, trial state and crash counts.
- **Confirmed in phase 0:** HAD_POR stays set after watchdog and ROM reboots, since `CHIP_RESET` still read `0x00010000`. So `CHIP_RESET` alone can't tell a warm reboot from a power-on. The marker caught every warm reboot, including one during a game, and the game kept running.

D9 (Decision): Wi-Fi starts only after the injection has finished.

- **Why:**
  - the CYW43 start-up loads a 225 KB blob from flash over the bus the payload DMA needs;
  - the radio needs at least 150 ms after power-up;
  - its current surge would land in the middle of the console's IPL read.
- **What it costs:** the W LED (on the CYW43) stays dark during the injection (D26). PicoBoot-DX starts the chip during the injection to blink the LED; the owner chose not to.
- **Warm reboots:** without an injection, Wi-Fi starts right away.

- **”Original IPL once” (D22):** no injection either, but the console reads its own IPL then, so Wi-Fi waits 1.5 s (phase 4 review).

D10 (Decision): a boot-time budget, measured before anything is built on it.

- **How:** the Pico measures the margin from ”PIO ready” to the console's first CS itself (D31). There is no scope.
- **What:**
  - PicoBoot-DX, as the baseline (phase 0);
  - our firmware behind the loader (phase 1).
- **Warm boards:** repeat on a warm board (upstream #148).
- **Phase 0 baseline** (PicoBoot-DX on the DOL-001): 164–168 ms from ”PIO ready” to the first CS, cold and warm. `main()` to ”PIO ready” takes about 0.3 ms.
- **Phase 1 result** (our firmware behind the loader, same console): 159.3–163.6 ms, cold and warm. The loader path costs **3–5 ms**; `main()` to ”PIO ready” is still about 0.3 ms.
- **The rule (set in phase 1, step 1.11):** everything before ”PIO ready” (loader, firmware start-up, boot control, payload choice) may add **at most 10 ms** over PicoBoot-DX, so the margin stays above about 155 ms on this console.
  - Today's cost is 3–5 ms, so later phases have about 5 ms left before the injection. Work that needs more, such as hashing a payload, happens after the injection or at upload time.
  - Every test build's report page shows the margin, so each phase's console tests check the rule.
- **If the loader is too slow:** the loader starts the PIO state machines itself before chaining, and the firmware takes them over. Not needed after phase 1.

D31 (Decision): the Pico measures its own boot timing.

- **The counter:** a third, passive PIO state machine watches CS. It doesn't touch the two injection programs.
  - It raises a PIO IRQ flag on the first CS falling edge, which the CPU (idle during the injection) timestamps.
  - It counts every CS pulse until the injection ends.
  - The DMA transfer count gives the number of 1 KB transfers delivered.
- **The margin:** the time of the first CS minus the time of ”PIO ready”.
  - A positive margin together with a successful boot proves the PIO was in time.
  - A short CS count shows missed pulses.
- **Reuse:** the same counters become the boot diagnostics (D22).
- **What it can't see:** the supply ramp before the RP2350 leaves reset. That doesn't matter, because the margin is measured from the Pico's own start.

D36 (Decision): guard the 224 skipped pulses against CS glitches.

- **The failure, found in phase 0:**
  - PicoBoot's occasional blue screens come from one false CS falling edge 42–44 ms after ”PIO ready”, when the console first drives CS.
  - The owner has also seen them with PicoBoot-DX up to v0.10 and with upstream PicoBoot.
  - `on_transfer` counts the false edge as one of the 224 skipped transfers, so the payload goes out one transfer early.
  - Rate: 2 of 56 cold boots, and both blue-screened.
- **Fix 1, a pull-up on CS:**
  - PicoBoot never sets the pad pulls, so CS keeps the RP2350's reset pull-down. That holds the undriven CS low (”selected”) for the console's first ~42 ms.
  - The firmware turns on the CS pull-up instead, before the PIO starts. Phase 0 had 0 failures in 50 boots that way.
  - It also avoids RP2350-E9 on A2 chips, where ”the pad pull-up still works”.
- **Fix 2, a clock-qualified count:**
  - `on_transfer` counts a CS low period among the 224 only once CLK rises in it **while CS is still low** (`jmp pin` on CS). A pulse without clocks merges with the next real transfer.
  - A CLK pulse while CS is high (EXI channel 0's clock is shared with memory card slot A and SP1) counts nothing; the phase 0 review found this case.
  - The 225th transfer's first address clock must still reach `clocked_output`; phase 1 checks this in the simulator.
  - A passive counter with this rule caught the glitch on hardware, and agreed with the raw count on every

good boot.

- **Diagnostics:** the D31 raw and clocked counters stay. The boot report flags a difference: a glitch the injection survived.
- **Proven in phase 1:** with PicoBoot's pad setting deliberately restored (a test mode), 2 power-up glitches in 40 boots, 36–37 ms after ”PIO ready”. Both were ignored, and Swiss started.
- **Known limit (phase 1 code review; the owner chose to keep the proven programs):**
  - ”CLK rises” is checked as ”CLK reads high” (`wait 1 gpio`), not as an edge.
  - If CLK already reads high when a false CS pulse comes, the pulse still counts. On A2 chips that could happen if the console drives CLK high at power-up and then releases it, since RP2350-E9 can then hold the pad at about 2.2 V.
  - Not seen: CLK was low in both glitches caught.
  - The fix, if it's ever needed, is one `wait 0 gpio CLK` before each of the two waits (`on_transfer` and the passive counter).
- **Compatibility:** the injection programs now differ from PicoBoot-DX v0.10. D4's compatibility covers the wiring, the payload format and the address, not the PIO code.

Boot flow:

```
power on -> boot ROM -> loader (flash offset 0)
  |- power-on?  HAD_POR set and no session marker: set marker, power_on = 1
  |- pick slot: pending trial slot, else the active slot (D12),
  |             else the other slot if this one keeps crashing
  |- boot info -> POWMAN scratch (power_on, slot, trial, reason)
  `- rom_chain_image(slot) -> firmware (slot A or B, mapped to 0x10000000)
       |- if power_on: 250 MHz; PIO + DMA inject payload 1, 2 or built-in;
       |               no flash writes; wait for disable_chip; lower the clock
       |- start the watchdog, then Wi-Fi (blob from Wi-Fi A or B)
       |- AP, DHCP, DNS, web UI; Wi-Fi auto-off (D20)
       `- trial bookkeeping (D12), USB payload check (D17)
```

5. Firmware updates

D11 (Decision): one update file per release, checked with SHA-256, not signed.

- **The file:** `picoboot-wifi-X.Y.Z-update.bin`, made in CI. It holds:
  - a header: magic, format version, firmware version, board, sizes, SHA-256 of each part;
  - the slot image;
  - optionally the Wi-Fi blob.
- **On the device:**
  - it streams the image into the inactive slot while hashing it, then reads it back and hashes again;
  - only then does it mark the slot as pending and reboot.
- **USB-only images:** a file that's meant only for USB (the install UF2) is refused.
- **No signing:** the Wi-Fi password is the gate (D19). The owner's own test builds install without extra steps. Signing (a private key in CI, about 10 KB of code) can come in a later release.
- **The format** is in update-format.md (phase 3). The owner chose:
  - the Wi-Fi blob always in the release file, written only when the paired area holds another one;
  - downgrades allowed, with a warning in the UI.
- **Compatibility:** the header names the flash layout and the lowest loader version the firmware needs. The loader changes only with a USB install (D24), so the Pico refuses an update its layout or loader doesn't fit, and says a USB install is needed.

D12 (Decision): a new firmware is on trial until it has proven itself.

- **Trial:** after an update the new slot is pending, and the old slot is still the fallback.
- **Crash:** if the new firmware crashes or hangs (watchdog) while on trial, the loader boots the old slot. The old firmware records ”trial failed” in boot control and shows it on the status page.
- **Keep:** the new firmware becomes the active one after one of these:
  - its first complete power-on injection, followed by Wi-Fi coming up;
  - the user pressing *Keep* on the firmware page.

- **A crash during the trial's first power-on boot:** that one console start goes to the original IPL. The firmware can't fall back and still inject in time (C1, C2). The next power-on uses the old firmware.
- **Crash loop:** a crash loop in the active firmware (N crashes in one power-on session) makes the loader try the other slot, if it holds a firmware that was kept before.
- **As built in phase 3** (step 3.4), with the owner's choices:
  - **N is 3** crashes in a row, and the fallback is **for good**: the firmware it falls back to becomes the active one, and the crashing slot is marked failed (D13 can switch back by hand).
  - **The loader remembers the session in POWMAN scratch,** since it never writes flash: the slot it started last, whether on trial, its crashes in a row, and which trials failed. Scratch survives warm reboots and is cleared at power-on, so every console session starts afresh.
  - **A crash is any reset the firmware didn't mean.** Before each reboot it means (the UI, an update), the firmware writes a clean-reboot word into scratch. A watchdog reset (8 s, fed in the injection's waits and the main loop) or a fault (the fault handler reboots at once) lacks it.
  - **The firmware records the news:** a failed trial (the slot becomes failed, nothing is pending), or the takeover after a crash loop. On keeping, the slot it replaces becomes kept as well, as a firmware that ran, so a later crash loop can fall back to it.
  - **Never a fallback:** a slot that's empty (perhaps half written), failed, or on trial isn't started in place of a slot without a valid image.
  - **No update while on trial:** the running firmware refuses one (`on_trial`), since the other slot holds the firmware to fall back to.
  - **The loader reports its version (2)** in boot info; the phase 3 firmware needs it, so it comes with one USB install (D11).
  - **After the phase 3 review:**
    - **Keeping is checked:** after a complete power-on injection, the firmware asks the Wi-Fi chip whether the access point is up (the owner's choice). A firmware on trial whose Wi-Fi doesn't come up reboots as a crash, so the loader goes back.
    - **Updates only from the active firmware,** into the other slot: never into the active slot, which would skip the trial. A firmware started in place of an active slot without an image becomes the active one.
    - **An empty slot is never started.** After a USB install, slot B counts as empty: the install doesn't touch it, so it may hold anything.
    - **An update writes the image's first sector last,** once the image's hash is right: that sector holds the block the boot ROM looks for, so a cut-off or damaged upload never leaves a slot the ROM would start.

D13 (Decision): switch freely between two good slots.

- The firmware page shows both slots: version, build, state (active / pending / failed / empty).
- *Boot slot X* writes boot control and reboots. Nothing is erased.
- **As built in phase 3:** a kept slot becomes the active one at once; any other slot with a firmware (from a USB install, unkept or failed) goes on trial first (D12). Going back to the active slot from a trial abandons the trial.

D14 (Decision): the Wi-Fi blob outside the slots, one copy per slot, at fixed addresses.

- **Pairing:** slot A uses Wi-Fi A and slot B uses Wi-Fi B. An update can then bring a new blob without touching the one the fallback firmware needs.
- **Hash:** each update file names the SHA-256 of the blob it needs.
  - If the paired Wi-Fi area holds a different blob, the device takes it from the update file, which always carries it (D11). Phase 3 dropped copying it from the other area.
  - Normally the blob doesn't change, and an update is only the ~170–200 KB slot image.
- **Loading:** the CYW43 driver reads the blob through the untranslated XIP window (`0x1c000000` + offset). This replaces the SDK's partition-table lookup, a hook of a few lines (sizes.md).
- **Which blob:** Wi-Fi only (225 KB), since Bluetooth is a non-goal.

6. Payloads

D15 (Decision): two payload slots; the user chooses which one boots.

- **Two complete payloads:** payload 1 (the USB address) and payload 2 each hold a full payload, e.g. Swiss and gekkoboot side by side.

- **Uploads:** an upload goes to the slot that isn't booting and switches to it only after it has been verified. The previous payload stays as a choice.
- **Power cuts:** no copying, so a power cut during an upload never touches the payload that boots.
- **Fallback:** the built-in gekkoboot is always a third choice, and boots when the chosen payload is invalid.

D16 (Decision): what the device accepts, and where `.dol` files are converted.

- **Payload UF2:** any family. The device takes one family's blocks at `0x10080000` onwards, so gekkoboot's RP2040-only UF2s work too, which they don't over USB on a Pico 2.
- **Raw payload:** the `IPLBOOT` … `PICO` image as it sits in flash.
- **Checks:** every upload is checked for header, footer, size (≤ 1.5 MB for payload 1; ≤ 1.5 MB − 4K for payload 2, D37) and 1 KB alignment before it can be chosen.
- **`.dol` files:** converted to a payload in the browser (JavaScript port of `tools/process_ipl.py`) and in the CLI tool (the Python original). The device doesn't convert them: DOL sections need random access, which a streamed upload doesn't give.

D43 (Decision): no loader for ordinary `.dol` files in the first release (the owner's choice, phase 4).

- **Today:** the converter (D16) takes only a `.dol` built as an IPL replacement: loaded at, and started from, `0x81300000`, where the boot ROM puts the IPL and jumps to it (gekkoboot, iplboot, Swiss's IPL build). Others are refused, e.g. Swiss's regular `swiss_r2119.dol`, which starts at `0x80397940`.
- **What it would be:** a small loader stub built for `0x81300000`, with the `.dol` appended. The stub copies the sections to their own addresses, clears the BSS and jumps to the entry point, as Swiss's PicoBoot build does in effect. The converter in the browser and the CLI would put the two together.
- **Status:** an option for later releases. It needs a GameCube toolchain (devkitPPC) for the stub, and console tests with a range of `.dol` files.

D42 (Decision): payload uploads, names and backups (phase 4, the owner's choices; details D15–D17, D22).

- **Names:** each payload has a short name you choose, up to 24 printable characters. The payload page pre-fills it from the file name (`swiss_r2119.dol` → `Swiss r2119`); it can be changed at the upload and renamed later.
  - **Why it costs a USB install:** the names live in boot control, which the loader reads. Boot control gets format 2, and the loader version 3.
  - **From then on:** the loader, and the firmware, read only the fields they know of any newer boot control record. Later formats only add fields at the end, so they need no new loader.
- **Where an upload goes:** into the payload slot that isn't the choice (D15); while the built-in gekkoboot is the choice, into payload 2, so that payload 1 keeps what a USB install put there (D17).
- **Delete:** only a payload that isn't the choice; choose another one first.
- **Backups:** plain downloads. The settings backup leaves the Wi-Fi password out, as the API never returns it (D41).

D17 (Decision): a payload installed over USB always boots next.

- **How it's detected:** boot control keeps a short fingerprint of payload 1: a CRC-32 of its header (with the size), of 64 words spread over it, and of its last 256 bytes (with the footer). It's quick enough to compare before the injection (C1). Builds of a payload share much of their start, so the samples tell another build of the same size apart (phase 4 review); a payload written again unchanged can't be told apart.
- **Blank boot control** (after a USB install of the firmware): payload 1 is the choice anyway; its fingerprint is recorded without a USB-install notice.
- **A delete or an upload** zeroes the payload's first page before boot control records it, so a power cut in the erase leaves no old header that would look like a USB install.
- **What happens:**
  - if the fingerprint differs, payload 1 was written over USB and is booted, whatever was chosen before;
  - after a complete injection, the firmware records the new fingerprint. Until then every power-on counts as the first one after the USB install, so the page says so at the console's power-on, not only at the Pico's own reboot on USB power after the drag-and-drop;
  - a choice, a rename or an upload over Wi-Fi records it too, so the user's choice holds;
  - the web page shows that it switched.

7. Wi-Fi and web

D18 (Decision): access point, captive portal, lwIP httpd, small static UI, JSON API.

- **Access point:**
  - `cyw43_arch_lwip_poll`, single core, everything in one main loop after the injection;
  - DHCP and DNS servers from pico-examples;
  - the DNS answers every name with `192.168.4.1`, so phones open the page by themselves.
- **Web server:** lwIP httpd.
  - The static UI is plain HTML, CSS and JavaScript, gzipped, under about 30 KB, built into the firmware.
  - Downloads (backups) stream straight from flash through httpd's custom files.
- **Uploads:** `fetch()` with `application/octet-stream`. lwIP httpd doesn't parse multipart, and `LWIP_HTTPD_POST_MANUAL_WND` holds the upload back while a sector is erased.
- **JSON API:**
  - `GET /api/status`;
  - `POST /api/firmware`;
  - `POST /api/payload`;
  - `POST /api/settings`;
  - `POST /api/boot` (switch slot or payload, "original IPL once");
  - `POST /api/reboot`;
  - `GET /api/backup/…`.

  The CLI tool uses the same API.

D19 (Decision): WPA2 with a fixed, documented default password.

- **Out of the box:** SSID `PicoBoot-XXXX`, where XXXX comes from the chip's unique ID, with a default password that's in the README.
- **Change prompt:** a banner on every page until the password is changed.
- **The Wi-Fi password is the only gate:** no separate web login and no image signing (D11).
- **Forgotten password:** reinstalling the USB install UF2 resets the settings.
- **What the gate means** (phase 5 security review):
  - Anyone with the password can install any firmware. A hostile one could do what any firmware can, including writing the RP2350's one-time-programmable memory, which a USB reinstall can't undo. Signing (D11) is what would close that.
  - Such a client can also wear out the flash by changing settings or choices over and over. A store that changes nothing writes nothing, but the rest is accepted: it needs the password, which already allows worse.
  - A web page elsewhere can read the GET answers through DNS rebinding, though it can't change anything (POSTs need `Host: 192.168.4.1`). So no GET answer holds the password or another secret (the owner's choice).
- **Details:** see D30.

D20 (Decision): Wi-Fi turns off after 10 minutes without a client.

- **When it runs:** the AP starts after every boot (after the injection, D9) and stays on while a client is connected.
- **Setting:** 2, 5, 10 or 30 minutes, or always on.
- **Why:**
  - it saves about 40 mA plus Tx bursts on the console's rail (C5);
  - it limits radio noise during play;
  - it limits exposure.
- **To reach it again:** power-cycle the console.
- **Timing:** the countdown starts when the Wi-Fi chip drops the last client. A device that leaves without saying goodbye stays on the chip's list until it times out, so Wi-Fi can go off about 1.5 minutes later than set (phase 2 bench check: 3.5 minutes for the 2-minute setting). The owner accepted this.

D21 (Decision): no home Wi-Fi in the first release.

- **What it would be:** station mode with mDNS (`picoboot.local`), falling back to the AP when the network can't be joined.

- **Status:** kept as an option for later releases. It costs about 10 KB and a settings page, which fits in the slots' margin (D6).

D30 (Decision): access point security details.

- **WPA2-AES only.** WPA3/WPA2 transition can come later, once phase 2 shows that phones and laptops connect reliably.
- **Default password:** `picoboot` (8 characters, the WPA2 minimum), documented in the README.
- **SSID:** `PicoBoot-` plus the last four hex digits of the chip's unique ID, so two boards in one room can be told apart.

8. Features and storage

D22 (Decision): the feature set of the first release.

Feature	Phase
Status: firmware version and build, active slot, the other slot's version and state, payload slots (name, size, SHA-256), uptime, temperature, Wi-Fi clients	2
Diagnostics of the last boot, and a log of the last 32 (D39): power-on or warm boot and why, loader decision, payload used and why, CS pulses seen, transfers delivered, injection complete or stalled (and after how many transfers), time to ”PIO ready”, time to Wi-Fi up	2
Settings: SSID, password, channel, country, Tx power, Wi-Fi auto-off, LED on/off	2
Firmware: upload, progress, verify, trial/keep, switch slots	3
CLI tool: status and firmware update first, then payload upload and `.dol` conversion	3–4
Payloads: upload, choose, delete, download	4
”Boot the original IPL once”: skip the injection at the next power-on, for changing the console's clock or language	4
Backup: download payloads, the firmware slot and settings	4

Home Wi-Fi (D21) is left out of the first release.

D23 (Decision): boot control and settings survive power loss.

- **Layout:** each area has two 4K sectors. Every write goes to the other sector, as a record with a sequence number and a CRC. The newest valid record wins, so a power cut during a write leaves the previous record.
- **When writes happen:** only after the injection, never in the loader (C6). Writes are rare (an update, a switch, a settings change), so wear doesn't matter.
- **Empty or invalid:** the loader treats empty or invalid boot control as ”slot A active, payload 1”, which is the state right after a USB install.

D40 (Decision): the settings, their values and defaults (details D22's settings).

- **Stored:** a record of 128 bytes in the settings area (D23), with spare bytes for later settings in the same format version. A USB install blanks the area, which means the defaults (D19).

Setting	Values	Default
SSID	1–32 printable ASCII characters	`PicoBoot-XXXX` (D30)
Password	8–63 printable ASCII characters (a WPA2 passphrase)	`picoboot` (D30)
Country	worldwide, or one of the 51 countries the CYW43 driver lists	worldwide
Channel	1–11	6
Tx power	low 5 dBm, medium 10 dBm, high 15 dBm, max (the chip's own limit); the owner's choice in step 2.6	medium
Wi-Fi off after	2, 5, 10 or 30 minutes, or never (D20)	10 minutes
LED	on, off (D26)	on

- **Channel 6:** the owner's choice. The driver's default, 3, overlaps both 1 and 6; channels 1, 6 and 11 don't overlap, and every country allows them.
- **Only 1–11:** the Americas don't allow 12 and 13, and a channel the country doesn't allow could keep the AP from starting.
- **Named Tx power levels:** the owner's choice, easier to pick on a phone than a dBm number. The default is below the maximum, since phase 0 found plenty of range.
- **A stored value that isn't allowed** falls back on its own default, and the other settings stay. The firmware checks every value before it stores one, so this only guards against bugs.

D41 (Decision): how a settings change takes effect.

- **At once, without a reboot.**
  - Tx power, LED and auto-off apply straight away.
  - A change to the SSID, password, channel or country restarts only the access point, about a second after the answer: the firmware keeps running, and the console is untouched. Phones drop off and join again.
- **Wi-Fi changes are on trial** until a client reaches the web server through the restarted access point. Phones' captive-portal probes do that by themselves when they join.
  - Then the change is kept.
  - If the firmware starts again before that, the previous Wi-Fi settings return, and the status says so.
  - **Why:** the Pico sits inside the console. Without the trial, a mistyped password means opening the console for a USB install (D19). With it, power-cycling the console is enough.
- **The password is write-only:** the API says only whether it is still the default (for the D19 banner).

9. Releases, install and tools

D24 (Decision): what a release contains.

- **`picoboot-wifi-X.Y.Z.uf2`:** the USB install, for a first install, recovery, or an upgrade from PicoBoot-DX.
  - It writes the loader, blank boot control and settings, Wi-Fi A and slot A, all at absolute addresses (rp2350-arm-s family, no partition table).
  - It doesn't touch the payloads.
- **`picoboot-wifi-X.Y.Z-update.bin`:** the OTA update (D11).
- **Tools** (Python standard library, as in PicoBoot-DX):
  - `.dol` → payload (raw and UF2);
  - update-file builder (CI);
  - CLI client for the JSON API.
- **Version:** semver, in the update header and in picotool binary info. Test builds are 0.N.0-dev (N = the phase) plus the git hash; a release is its plain version, and the first is 1.0.0 (D35).
- **How it's made** (phase 5): a tag `vX.Y.Z` that matches the version in `CMakeLists.txt` starts `.github/workflows/relea`
  It builds and tests as CI does, and makes a draft release with the USB install, the update file, the CLI (which holds the `.dol` converter) and `SHA256SUMS`, its notes from `docs/releases/X.Y.Z.md`. The owner checks the draft and publishes it.

D29 (Decision): license GPL-2.0-only. PicoBoot-DX and the gekkoboot code it bundles are GPL-2.0, and this project reuses both.

D35 (Decision): no pre-releases. Test builds come from CI artifacts on the pushed phase branch, or from a local Docker build with the same SDK and toolchain as CI. The first GitHub release is 1.0.0.

10. Build and test

D25 (Decision): build and test setup.

- **Toolchain:** Pico SDK 2.3.1, picotool 2.3.1, CMake. The targets are the loader, the firmware, the Wi-Fi blob image, the install UF2 and the update file.
- **CI** (GitHub Actions, ubuntu-26.04):
  - builds;
  - Python tests for the tools;
  - the PIO simulator from PicoBoot-DX against the real PIO programs;

  - a layout check of each ELF against the flash map;
  - host-compiled C tests for the loader's slot decision and the boot-control records.
- **Hardware tests:** the owner, on a real GameCube.

D26 (Decision): LED.

- **During the injection:** dark (D9).
- **After it:**
  - three slow blinks if it completed (as in PicoBoot-DX);
  - fast blinks if it stalled, for as long as Wi-Fi is on (the owner's choice in phase 2; PicoBoot-DX also keeps blinking after a stall);
  - on while the AP is up, off when Wi-Fi is off.
- **Setting:** LED off.

D27 (Decision): the clock during the injection is fixed after phase 1.

- **Start:** 250 MHz, as in PicoBoot-DX, as a build setting.
- **Upstream reports:**
  - some Picos take the overclock badly (#146);
  - a Pico 2 W batch fails when warm, blamed on slow flash (#148).
- **Phase 1 measures:**
  - the DMA's margin against the console's EXI data rate;
  - warm-board behaviour.
- **Then:** the default clock and the flash clock divider during the payload DMA are set from those numbers.
- **Set in phase 1 (step 1.11): 250 MHz** during the injection and **flash divider 3**, both build settings.
  - The firmware sets up quad I/O flash reads itself (embedded `boot2_w25q080`); after the loader's chain, the boot ROM leaves the slowest serial mode (0.06 MB/s).
  - Divider 3 gives an 83 MHz flash clock at 250 MHz, below the W25Q32RV's 133 MHz.
  - The payload DMA through the uncached window reads 18 MB/s at 125 MHz, about 5.5 times the console's ~3.3 MB/s, and roughly double that at 250 MHz.
  - 87 console boots in phase 1, cold and warm, had no stall or fault.
  - If a board takes 250 MHz or warm flash badly (upstream #146, #148), a lower clock or a slower divider is the first thing to try.

11. Phases

D28 (Decision): hardware checks first, then five phases.

- **Phase 0 first,** before the core code. Its results can still change this design.
- **Phases 1–5** as listed below.

Phase	Content	Done when
0. Hardware checks	A measurement build of PicoBoot-DX with the D31 counter and the D32 report page, inside the closed console; the D33 rail check	Radio works on 3V3; current and rail sag measured; range known; boot-timing margin measured
1. Core	Loader, layout, firmware with the injection, the power-on rule, boot info, the D31 counter, the Wi-Fi blob hook and report page (D32), install UF2, CI	The console boots a payload through the loader on a cold and a warm board; a warm reboot doesn't inject; the boot-time budget is met (D10); the clock is set (D27)
2. Wi-Fi	AP, captive portal, status and diagnostics, settings, auto-off, LED	A phone reaches the status page from a closed console
3. Firmware OTA	Update file, slot writes, trial and fallback, switching, Wi-Fi blob pairing, CLI tool (status, update)	An update, a failed trial and a switch all work in the console without USB
4. Payload OTA	Payload slots, upload formats, browser `.dol` conversion, USB rule, ”original IPL once”, backups, CLI payload commands	Swiss and gekkoboot uploaded and chosen over Wi-Fi

Phase	Content	Done when
5. Release 1.0	README and install guide, whatever phases 0–4 show is needed	Owner's hardware test passes on a fresh install and on an upgrade from PicoBoot-DX

Firmware OTA comes before payload OTA: once it works, every later test runs without opening the console.

D32 (Decision): a minimal Wi-Fi readout comes early.

- **Why:** the measurements have to come out of a closed console.
- **Phase 0:** the PicoBoot-DX measurement build starts an access point after the injection. It has a plain-text report page and two load-test URLs: a large download (Tx load) and an upload sink (Rx load).
- **Phase 1:** the Wi-Fi blob hook (D14) and the same page move forward from phase 2, since the blob doesn't fit inside a 256K slot.

D33 (Decision): the rail check without a scope.

- **Average current:** measured with the meter with the AP off, idle, with a client connected and during the large download.
- **Rail sag:** checked with a stress test. The large download runs at full Tx power while Swiss loads a game from the SD2SP2, with the HDMI adapter on. Console glitches, video artefacts, Pico brown-outs (`HAD_B0R`) and Wi-Fi drops are recorded.

D34 (Decision): the DOL-001 is the test console.

- **Setup:** all hardware tests run on the owner's DOL-001, with the SD2SP2 and the HDMI adapter.
- **DOL-101:** the README says it's expected to work (same IPL path as PicoBoot) but is untested.

12. Risks and hardware checks

#	Risk	Check	If it goes wrong
R1	The loader makes the boot too slow	Phase 0: 164–168 ms margin on the DOL-001; phase 1 measures the loader	The loader starts the PIO itself, or slot A boots without a loader on the power-on path
R2	Tx bursts pull the console's 3.3 V rail down	Phase 0: 3.15 V average with the radio on; stress test clean	5 V wiring recommended (D38); lower Tx power by default
R3	No usable range from a closed console	**Closed** in phase 0: −42 dBm at 5 m	—
R4	Warm boards or slow flash break the injection	Phase 0: warm boots fine (the board only reached ~22 °C on its sensor); D27, phase 1	Lower clocks or a slower flash divider during the injection
R5	Wi-Fi disturbs the EXI bus or the video	Phase 0 stress test (SD2SP2, HDMI adapter): nothing seen; phase 2 again	Wi-Fi auto-off by default (D20); lower Tx power
R6	HAD_POR or scratch registers behave differently from the datasheet	**Closed** in phase 0: HAD_POR survives warm reboots; the D8 marker covers it	—
R7	A future CYW43 blob outgrows 240K	Each SDK update	Layout change through a USB install
R8	A CS glitch at power-up shifts the injection	D36; phase 1 in the simulator and on hardware	—